

Implementação de Redis para Sistemas Distribuídos

Christian Aguiar Plentz¹, João Roberto Lemos Fidellis²

¹Ciência da Computação – Universidade do Vale do Rio dos Sinos (UNISINOS)
Caixa Postal 275 – 93022-750 – São Leopoldo – RS – Brazil

{plentzchristian, JRFIDELLIS}@edu.unisinos.br

Resumo. *Esse artigo demonstra o funcionamento no redis em um sistema de Login que faz o armazenamento da sessão através do redis.*

Abstract. *This article demonstrates the inner workings of redis in a Login system which makes use of Session storage through redis.*

1. Estratégia

Objetivo: utilizar Redis como uma camada leve de armazenamento de estado (sessões) e cache para permitir sessões escaláveis e rápidas sem persistir estado sensível no servidor principal.

Principais decisões:

- Usar Redis para sessões (via `connect-redis`) — sessão centralizada, compartilhada entre instâncias do backend.
- Executar Redis em container (Docker) para facilitar reprodução do ambiente.
- Proteger acesso ao Redis quando o porto do container é publicado no host (usar autenticação com `--requirepass` e publicar apenas em `127.0.0.1` durante desenvolvimento).
- Configurar o backend para ler a URL do Redis via variável de ambiente `REDIS_URL` (padrão: `redis://:PASSWORD@127.0.0.1:6379`).

2. Arquitetura

A arquitetura adotada é simples e eficaz para o propósito do projeto: Frontend (browser) → Backend (Express) → Redis (sessões). Abaixo está um diagrama simplificado.

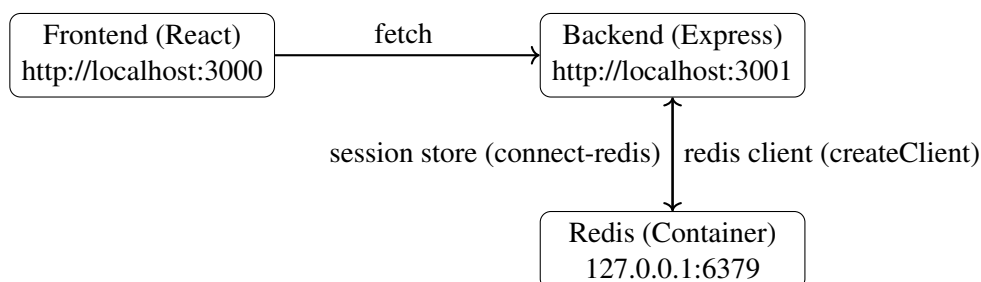


Figure 1. Visão arquitetural simplificada

Observações específicas do projeto:

- O backend atual define a URL do redis em `server.js` (exemplo: `redis://:your-strong-dev-pass@localhost:6379`). Verifique a variável de ambiente `REDIS_URL` em produção/testes.
- As sessões são salvas com prefixo `sess:` por padrão (`connect-redis`). Use comandos `SCAN` / `GET` no `redis-cli` para inspecionar.

3. Infraestrutura

Recomenda-se executar Redis via Docker Compose durante o desenvolvimento. Exemplo mínimo (adaptado do repositório):

```
# backend/docker-compose.yml (trecho)
version: '3.8'
services:
  redis:
    image: redis:7-alpine
    command:
      - redis-server
      - --bind
      - 0.0.0.0
      - --requirepass
      - "<your-dev-pass>"
      - --protected-mode
      - "yes"
    ports:
      - "127.0.0.1:6379:6379"
```

Notas de segurança:

- Em desenvolvimento, publicar em 127.0.0.1 impede acesso direto da rede externa.
- Em produção, evite publicar Redis ao público; prefira networking interno (sem ports:) e autenticação forte.
- Habilite ACLs/TLS em ambientes expostos.

4. Metodologia de testes (protótipos)

Objetivo: validar comportamento de sessões, latência e tolerância a falhas para o uso de Redis. A estratégia de testes segue três níveis:

4.1. Testes unitários

- Testes do código do backend (rotas, validação) com mocks do Redis. Use bibliotecas de testes do Node.js (Jest, Mocha). - Mock do client Redis para verificar que a função de criação de sessão chama o método `req.session.save` corretamente.

4.2. Testes de integração (local)

- Subir Redis via docker-compose e executar testes que fazem requests reais ao backend.
- Verificar que, após login, existe uma chave `sess:*` no Redis. Comandos úteis:

```
# check keypace
redis-cli -h 127.0.0.1 -p 6379 -a '<your-dev-pass>' INFO keypace
# scan sessions
redis-cli -h 127.0.0.1 -p 6379 -a '<your-dev-pass>' \\\
--scan --pattern 'sess:*
```

4.3. Testes de carga / protótipo

Cenários:

- 1) Conexões concorrentes de login: medir latência e throughput quando N usuários tentam logar simultaneamente (por exemplo, N=100, 500, 1000).
- 2) Persistência de sessão: conectar, confirmar sessão, reiniciar backend e confirmar que a sessão persiste em Redis (se esperado).
- 3) Falha e recuperação do Redis: simular reinício do container Redis e verificar comportamento do backend (reconexão, erros de sessão).

Ferramentas de teste:

- Artillery (node) ou hey / wrk para carga HTTP.
- redis-benchmark para testes específicos de throughput em Redis.

Exemplo simples de comando Artillery (fluxo de login):

```
# Exemplo (instale artillery globalmente: npm i -g artillery)
# arquivo simple-login.yml
config:
  target: "http://localhost:3001"
  phases:
    - duration: 60
      arrivalRate: 20
scenarios:
  - flow:
    - post:
      url: "/api/auth/login"
      json:
        username: "user"
        password: "user123"
```